# Hands-on session: Molecular Dynamics simulations of a LiFSI/DME electrolyte using GROMACS

**Format:** two 3-hour hands-on sessions
**Platform:** GitHub Codespaces for GROMACS/analysis; VMD is run locally.
**System under study:** 1 M LiFSI in DME electrolyte
**Main objective:** run a molecular dynamics (MD) simulation and extract structural and transport information from the trajectory.

## 1. Activity context: GitHub and GitHub Codespaces

GROMACS is a command-line molecular-dynamics package. Running simulations normally requires a suitable software installation and access to sufficient computing resources. This activity avoids local GROMACS installation by using GitHub Codespaces, while VMD is used separately on the local computer for molecular visualization.

### 1.1 GitHub and the wrokshop repository

GitHub is an online platform for storing and sharing software projects in repositories. The course repository contains the input structures, force-field files, simulation parameters, analysis material, and a development-container configuration.

A free personal GitHub account can be created at https://github.com/signup by providing an email address, password, and username, followed by email verification. After signing-in, the course repository can be opened from the following link:

https://github.com/rozas-ib/MD-electrolyte-workshop

### 1.2 What is a Codespace?

A GitHub Codespace is a remote Linux development environment hosted in the cloud. The browser displays a Visual Studio Code interface connected to that remote machine. Commands typed in the terminal therefore run on the remote Codespace rather than on the local computer.

The prepared repository defines the software environment. Each participant creates an independent Codespace from that repository, so GROMACS, Gnuplot, Python, MDAnalysis, and the other configured tools are available without local installation. Files generated during the practical are stored inside each participant Codespace until they are downloaded, committed to Git, or the Codespace is deleted.

### 1.3 Create and open the course Codespace

1. Sign in to GitHub and open the course repository.

2. Select Code > Codespaces > Create codespace on main (or use the Codespaces launch link supplied by the instructor).

3. Wait for the browser-based Visual Studio Code window to open and for the development container to finish starting.

4. Open Terminal > New Terminal. The terminal prompt should point to the repository inside /workspaces/.

5. Confirm that GROMACS is available by running the command below.

```
gmx --version
```

### 1.4 Free usage and Codespace lifetime

GitHub Free personal accounts include 120 Codespaces core-hours and 15 GB-month of storage per month. Codespaces compute is counted in core-hours: one wall-clock hour on a 2-core machine consumes 2 core-hours. Verified students with the GitHub Education benefit may receive the GitHub Pro allowance of 180 core-hours per month.

A Codespace does not run permanently. By default, it automatically stops after 30 minutes without activity, and a single running session can last for a maximum of 12 hours.

Stopping a Codespace does not delete the files. A stopped Codespace can normally be reopened for up to 30 days, after which GitHub may automatically delete it. Saved Codespaces can be accessed again from https://github.com/codespaces.

To avoid using the monthly computing allowance unnecessarily, the Codespace should be stopped at the end of each laboratory session. Computing time is only consumed while the Codespace is running, although stored files continue to use storage space while the Codespace remains saved.

## 2. Linux terminal introduction

The Codespace terminal is a text interface for running programs. A prompt may look similar to:

```
@username -> /workspaces/MD-electrolyte-workshop $
```

The $ symbol marks the shell prompt; commands are entered after it.

**Useful Terminal Commands**

| Command | Description |
|---|---|
| `pwd` | Shows the current working directory. |
| `ls` | Lists files and directories in the current folder. |
| `ls -lh` | Lists files with detailed information and human-readable file sizes. |
| `cd session-1` | Moves into the session-1 directory. |
| `cd ..` | Moves one directory upward. |
| `head filename` | Shows the beginning of a text file. |
| `tail filename` | Shows the end of a text file. |
| `tail -f filename` | Continuously follows a file while it is being written. Press Ctrl+C to stop following the file. This does not stop a simulation running separately in the background. |
| `less filename` | Opens a text file for reading without editing. Use arrow keys or Page Up/Page Down to navigate, /text to search, n for the next match, and q to quit. |

**Terminal Text Editors**

Three terminal text editors are installed in the Codespace: nano, vi, and emacs.

| Editor | Command | Useful keyboard shortcuts |
|---|---|---|
| nano | `nano filename` | Ctrl+O: Save file; Enter: Confirm filename; Ctrl+X: Exit; Ctrl+W: Search for text. |

| | | |
|---|---|---|
| vi | `vi filename` | i: Enter insert mode; Esc: Return to command mode; :w: Save; :q: Quit; :wq: Save and quit; :q!: Quit without saving. |
| Emacs | `emacs -nw filename` | Ctrl+X, Ctrl+S: Save; Ctrl+X, Ctrl+C: Exit; Ctrl+S: Search; Ctrl+G: Cancel current command. The -nw option keeps Emacs inside the terminal. |

**Useful Keyboard Shortcuts and Command-Line Conventions**

| Shortcut / Convention | Description |
|---|---|
| Tab | Automatically completes a filename or command. |
| Up arrow | Recalls previously executed commands. |
| Ctrl+C | Interrupts the command currently running in the foreground. |
| Case sensitivity | Commands and filenames are case-sensitive. For example, DME and dme are different. |
| Spaces | Separate command-line arguments. |
| Single quotes ('...') | Treat enclosed text as one argument. This is particularly important for GROMACS selections. |

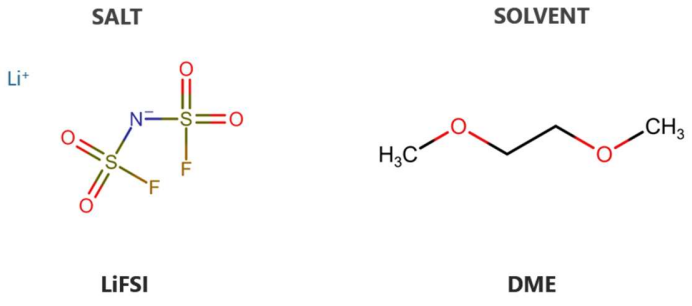
## 3. System to simulate

1 M LiFSI in DME solvent is equivalent to a LiFSI:DME molar ratio equal to 1:9.To build a box containing **100 DME molecules,** the required number of LiFSI formula units is therefore

$$N_{\mathrm{LiFSI}} = \frac{100}{9} = 11.11 \approx 11.$$

Because one LiFSI formula unit contains one $Li^+$ and one $FSI^-$, the simulation box will contain:

| Species | Number |
|---|---|
| DME | 100 |
| $FSI^-$ | 11 |
| $Li^+$ | 11 |




## 4. Repository structure

The course repository is organized into two sessions:

```
.
|-- LICENSE
|-- README.md
|-- references/
|   `-- README.md
|-- session-1/
|   |-- input/
|   |   |-- dme.gro
|   |   |-- fsi_freq.gro
|   |   `-- li.gro
|   |-- mdp/
|   |   |-- min.mdp
|   |   |-- npt_press.mdp
|   |   |-- npt_eq.mdp
|   |   `-- nvt.mdp
|   `-- topology/
|       |-- dme.itp
|       |-- ffbonded.itp
|       |-- ffnonbonded.itp
|       |-- forcefield.itp
|       |-- fsi.itp
|       |-- li.itp
|       `-- topol.top
`-- session-2/
    |-- analysis/
    `-- trajectory/
        |-- nvt.gro
        |-- nvt.xtc
        |-- nvt.tpr
        `-- nvt.edr
```

## 5. Before starting: what are these GROMACS files?

Several GROMACS file types are used throughout the activity.

| File | Purpose |
|---|---|
| .gro | Atomic coordinates, atom/residue names, and simulation-box dimensions |
| .itp | Molecular topology/force-field information for a molecule or part of a force field |
| .top | Main system topology: includes force-field files and specifies how many molecules are present |
| .mdp | Human-readable simulation parameters |
| .tpr | Binary GROMACS run-input file generated by gmx grompp |
| .xtc | Compressed trajectory containing coordinates as a function of time |
| .edr | Energy/thermodynamic data such as temperature, pressure, density, and potential energy |
| .log | Text log written by gmx mdrun |
| .xvg | Text data produced by GROMACS analysis tools; can be plotted with Gnuplot |

The basic workflow is:

```
coordinates + topology + simulation parameters
                ↓
            gmx grompp
                ↓
              .tpr
                ↓
            gmx mdrun
                ↓
```

```
trajectory + energies + log
          ↓
       analysis
```

# SESSION 1 - Build, equilibrate, and start the MD simulation

## 1. Enter the Session 1 directory

From the repository root:

```
cd session-1
```

Check the current working directory:

```
pwd
```

List the files:

```
ls
```

The following directories should be present:

```
input  mdp  topology
```

Inspect each directory:

```
ls input
ls mdp
ls topology
```

## 2. Build the starting electrolyte box

To simulate a 1 M LiFSI electrolyte in DME using MD, a representative sample of the bulk liquid must first be constructed inside a simulation box. Periodic boundary conditions (PBC) are applied to this box so that molecules leaving through one side of the simulation cell re-enter through the opposite side. In this way, the relatively small number of molecules contained in the simulation box can approximate the behavior of a much larger bulk liquid while avoiding artificial interfaces.

To introduce the $Li^+$, $FSI^-$, and DME species into the simulation box, an initially oversized and therefore unphysical volume is used. This starting volume is not intended to reproduce the experimental density of the electrolyte. The `gmx insert-molecules` tool performs essentially a geometrical packing operation: it attempts to place molecules within the available volume while avoiding excessive atomic overlap, but it does not account for the preferred chemical interactions or equilibrium organization of the liquid. If the molecules were inserted directly into a box corresponding to the experimental liquid density, the limited free space could prevent successful insertion or generate severe atomic overlaps.

For this reason, GROMACS will initially insert 100 DME molecules into a 4 × 4 × 4 $nm^3$ box. This volume is intentionally much larger than the equilibrium volume expected for the electrolyte and provides sufficient free space to accommodate all the molecular species required for the system. After inserting the DME molecules, $FSI^-$ anions and $Li^+$ ions are added to complete the desired composition. The resulting low-density configuration is subsequently energy-minimized and

compressed during the NPT simulations, allowing the box volume and density to evolve toward physically meaningful equilibrium values.

Insert 100 DME molecules:

```
gmx insert-molecules -box 4 4 4 -ci dme.gro -nmol 100 -o dme_box.gro
```

Breakdown of the command:

- gmx insert-molecules: GROMACS molecular insertion tool.
- -box 4 4 4: create a 4 x 4 x 4 nm simulation box.
- -ci input/dme.gro: molecule that will be inserted.
- -nmol 100: number of DME molecules.
- -o dme_box.gro: output coordinate file.

Insert 11 $FSI^-$ anions

```
gmx insert-molecules -f dme_box.gro -ci fsi_freq.gro -nmol 11 -o dme_fsi_box.gro
```

Here -f dme_box.gro tells GROMACS to preserve the existing DME molecules and insert FSI- into that box.

Insert 11 $Li^+$ ions

```
gmx insert-molecules -f dme_fsi_box.gro -ci li.gro -nmol 11 -o dme_fsi_li_box.gro
```

The final coordinate file for the complete initial system is:

```
dme_fsi_li_box.gro
```

Check the last line:

```
tail -n 1 dme_fsi_li_box.gro
```

It should contain box dimensions close to:

```
4.00000  4.00000  4.00000
```

The following strategy is used to simulate the electrolyte:

```
large, low-density box
        ↓
insert all molecules successfully
        ↓
energy minimization
        ↓
NPT simulation
        ↓
box contracts and approaches the liquid density
        ↓
NVT production run
```

## 3. Check the topology before running MD simulations

The coordinate file contains atoms, but GROMACS also needs to know their force-field parameters and molecular connectivity. That information is supplied through topology/topol.top and the included .itp files.

Inspect the topology:

```
less topology/topol.top
```

The [ molecules ] section should contain molecule counts consistent with the newly created coordinate file.

## 4. Energy minimization

The randomly inserted configuration is not physically equilibrated and may contain unfavorable close contacts. Energy minimization removes the worst energetically not favorable contacts before molecular dynamics begins.

The minimization stage does not represent physical time evolution; its purpose is to reduce large forces and remove severe overlaps before molecular dynamics starts.

No thermostat or barostat is active during energy minimization, because temperature and pressure are thermodynamic concepts associated with the subsequent molecular-dynamics stages.

### 4.1 Generate the minimization TPR file

```
gmx grompp -f min.mdp -c dme_fsi_li_box.gro -p topol.top -o min.tpr
```

gmx grompp combines:

```
min.mdp                  simulation settings
+ dme_fsi_li_box.gro     coordinates
+ topol.top              force field and system composition
-------------------------------------------------------------
= min.tpr                binary run input
```

Read the terminal output. **Errors must be fixed before continuing. Do not use -maxwarn simply to suppress a warning.**

### 4.2 Run minimization in the background

The simulation is launched in the background so that the terminal remains usable while mdrun is running.

Run:

```
nohup gmx mdrun -v -deffnm min -c min_out.gro > min_run.log 2>&1 &
```

What happened?

- gmx mdrun: runs the simulation.
- -deffnm min: uses min as the common filename prefix.
- -c min_out.gro: writes final coordinates to min_out.gro.
- > min_run.log: saves terminal output to a text file.
- 2>&1: puts error messages in the same file.
- &: runs the command in the background.
- nohup: helps the process continue if the terminal itself is closed.

The shell prints a job/process number, after which additional terminal commands can be entered.

Check background jobs in the same terminal:

```
jobs -l
```

Follow the output:

```
tail -f min_run.log
```

Press **Ctrl+C** to stop watching the log. This stops only tail; the minimization continues in the background.

Check whether GROMACS finished:

```
tail -n 20 min.log
```

Do not start the NPT simulation until minimization has finished successfully.

# 5. VMD introduction: inspect the starting and minimized structures

VMD runs locally, outside the Codespace. A first structural inspection can be performed as soon as the initial box and the minimized structure are available.

## 5.1 Download the structures

From the Codespaces file explorer, download (right click on the file > download…) the following files to the local computer:

```
dme_fsi_li_box.gro
min_out.gro
```

The first file contains the randomly packed starting box; the second contains the coordinates after energy minimization.

## 5.2 Load a structure in VMD

1. Start VMD and select File > New Molecule.

2. Browse to dme_fsi_li_box.gro and select Load.

3. Repeat the procedure for min_out.gro so that both structures are available as separate molecules.

4. The molecule list in the VMD Main window can be used to show or hide each structure and compare the geometry before and after minimization.

## 5.3 Molecular representations

Open Graphics > Representations. The Selected Atoms field defines which atoms are displayed; Drawing Method defines the graphical style; Coloring Method defines the color scheme.

Example selections include **name Li** for Li+, **resname FSI** for FSI-, and **resname DME** for DME.

## 5.4 Display the periodic simulation box

Open Extensions > Tk Console and enter:

```
pbc box
```

The box outline represents the PBC simulation cell. Only static structures are inspected at this stage; trajectory playback and movie generation are introduced in Session 2.

## 6. NPT pressurization: compress the oversized box

Once the minimization step is completed, the artificially big box should be compressed to obtain a physical and meaningful system configuration.

The initial 4 nm box is much too large for the desired liquid density. In an NPT simulation, both temperature and pressure are controlled, and the simulation box can change volume.

For the NPT pressurization step a T = 10 K and P = 5 bar is used. This pressurization stage is a preparation step and does not represent the final target thermodynamic state. The subsequent NPT equilibration stage changes the reference conditions to 298 K and 1 bar.

Generate the TPR:

```
gmx grompp -f npt_press.mdp -c min_out.gro -p topol.top -o npt_press.tpr
```

Run in the background:

```
nohup gmx mdrun -v -deffnm npt_press -c npt_press_out.gro > npt_press_run.log 2>&1 &
```

After completion, inspect the box dimensions:

```
tail -n 1 npt_press_out.gro
```

## 7. NPT equilibration

After the initial compression, an additional NPT stage allows the density and liquid structure to equilibrate.

For the NPT equilibration step a T = 298.15 K and P = 1 bar is used. During this stage the initially oversized simulation box should approach a stable volume and density appropriate for the modeled liquid.

Generate the TPR:

```
gmx grompp -f npt_eq.mdp -c npt_press_out.gro -p topol.top -o npt_eq.tpr
```

**Speeding up the simulation using multiple CPU cores**

The NPT equilibration step is longer than the previous stages, so it is useful to take advantage of the multiple CPU cores available in the Codespace.

First, check how many CPU cores are available:

nproc

For example, if the command returns 2 the Codespace provides two CPU cores.

GROMACS can distribute the MD calculations over several CPU threads. The number of threads can be specified with the -nt option:

```
gmx mdrun -v -deffnm npt_eq -c npt_eq_out.gro -nt 2
```

Therefore, to run the NPT equilibration in the background using two CPU cores:

```
nohup gmx mdrun -v -deffnm npt_eq -c npt_eq_out.gro -nt 2 > npt_eq_run.log 2>&1 &
```

Once the NPT equilibration step is completed, proceed to check the density of the electrolyte system. from the energy file (napt_press.edr and npt_eq.edr)

```
gmx energy -f npt_press.edr/npt_eq.edr -o density.xvg
```

The file density.xvg contains density as a function of time.

## 8. Evaluate and plot the equilibrated density

Start Gnuplot:

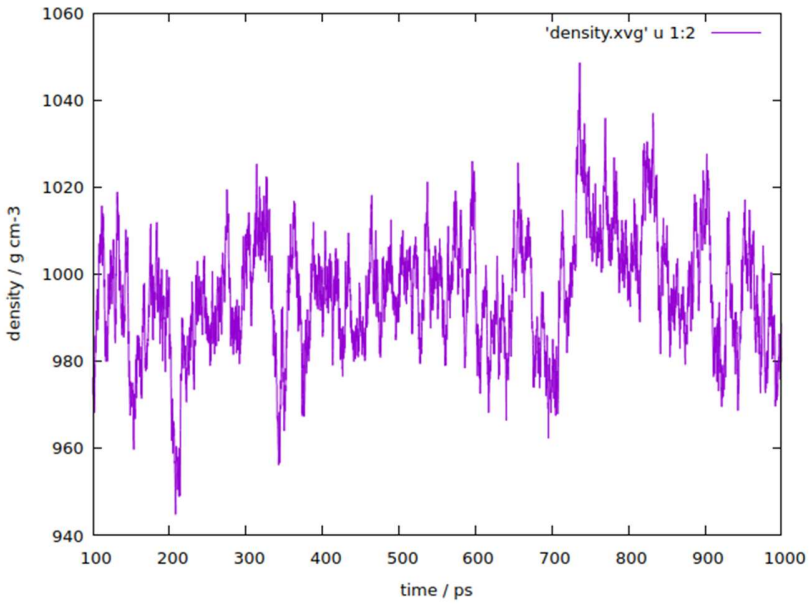
```
gnuplot
```

At the gnuplot> prompt enter:

```
set terminal pngcairo enhanced size 1200,800
set output "density.png"
set xlabel "Time (ps)"
set ylabel "Density (kg/m^3)"
plot "density.xvg" using 1:2 with lines title "Density"
unset output
exit
```

In the Codespaces file explorer, click density.png to open it.

Example:




Initially, during the npt_press step the density may change rapidly while the box adjusts. Later, during the npt_eq step it should fluctuate around a relatively stable mean.

To extract the average value, do not average over the initial transient, choose an equilibration time, t_eq, after which the density appears stationary.

For example, if the plot appears equilibrated after 100 ps:

```
gmx energy -f npt_eq.edr -o density_eq.xvg -b 100
```

GROMACS prints the average density and fluctuations in the terminal.

### Compare the equilibrated density with literature

Compare the equilibrated density with reported values for related LiFSI/DME electrolytes. One useful reference is Younesi, R. and Bardé, F., "Electrochemical performance and interfacial properties of Li-metal in lithium bis(fluorosulfonyl)imide based electrolytes," Scientific Reports 7, 15925 (2017), https://doi.org/10.1038/s41598-017-16268-7.

## 9. NVT production simulation

Once a reasonable liquid density has been obtained, the production trajectory is run at constant number of particles, volume, and temperature (NVT) for structural and transport analysis.

The NVT trajectory is the trajectory used for the RDF, MSD, diffusion, and conductivity exercises.

Generate the TPR:

```
gmx grompp -f nvt.mdp -c npt_eq_out.gro -p topol.top -o nvt.tpr
```

Run the simulation in the background:

```
nohup gmx mdrun -v -deffnm nvt -c nvt_out.gro > nvt.run.log 2>&1 &
```

A completed NVT simulation should produce at least: `nvt.tpr, nvt.xtc, nvt.edr, nvt.log and nvt_out.gro.` If it does not finish, session-2 folder contains a reference trajectory so that everyone can complete the analysis.

Important note: a short simulation as the obtained at the end of the session is useful for learning the workflow, but is usually too short for high-quality transport statistics. The supplied reference trajectory should therefore be preferred for quantitative comparison if it is substantially longer.

## 10. Post-NVT equilibration and stability checks

Before structural and transport properties are calculated, the trajectory should be checked for sampling of a stationary equilibrated state rather than the transient response to the starting configuration.

### 10.1 RMSD

The root-mean-square deviation (RMSD) measures how different a structure at time $t$ is from a reference structure. In a mass-weighted form,

$$\mathrm{RMSD}(t) = \left[\frac{1}{M}\sum_i m_i \left|\mathbf{r}_i(t) - \mathbf{r}_i^{\mathrm{ref}}\right|^2\right]^{1/2},$$

where $m_i$ is the mass of atom $i$, $M = \sum_i m_i$, and $\mathbf{r}_i^{\text{ref}}$ is its position in the reference structure.

RMSD is widely used for proteins, crystals, and other systems in which atoms remain associated with a persistent reference structure.

Calculate a demonstration RMSD for the NVT trajectory:

```
gmx rms -s nvt.tpr -f nvt.xtc -o rmsd_system.xvg
```

GROMACS requests two selections:

1. the group used for the least-squares fit;
2. the group used for the RMSD calculation.

For this demonstration, select **System** for both.

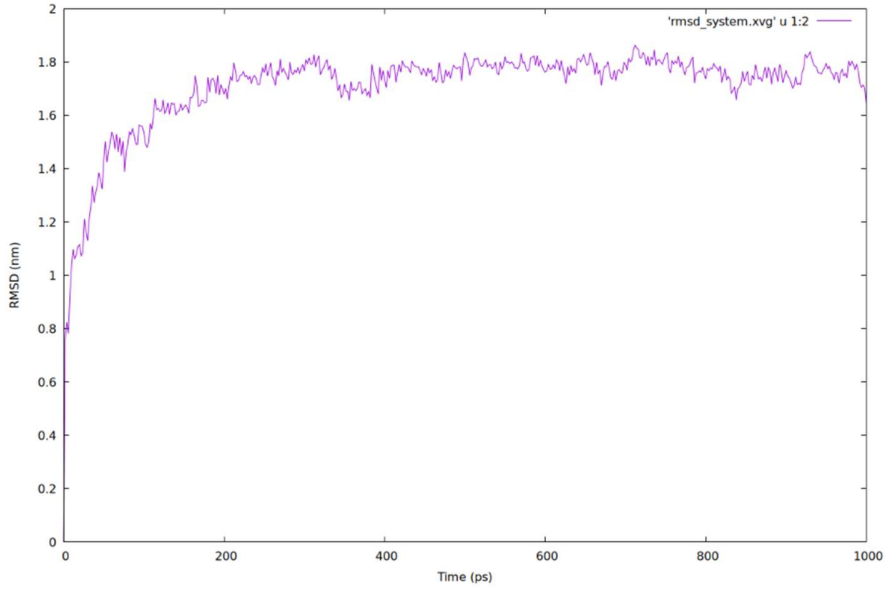
Plot it:

```
gnuplot
set terminal pngcairo enhanced size 1200,800
set output "rmsd.png"
set datafile commentschars "#@"
set xlabel "Time (ps)"
set ylabel "RMSD (nm)"

pl 'rmsd_system.xvg' u 1:2 w l
unset output

exit
```

Example:




For a liquid electrolyte, RMSD is not sufficient evidence by itself that the electrolyte simulation has equilibrated is not sufficient. Here, it is included to introduce the concept and the gmx rms command.

In a liquid, a more useful question is, have the statistical properties of the trajectory become stationary?

## 10.2 Temperature and potential energy

Use gmx energy to inspect basic NVT observables.

Temperature:

```
gmx energy -f nvt.edr -o nvt_temperature.xvg
```

When prompted, type Temperature, then finish the selection with 0.

Potential energy:

```
gmx energy -f nvt.edr -o nvt_potential.xvg
```

Select Potential, then 0.

Plot both separately:

```
gnuplot
set terminal pngcairo enhanced size 1200,800
set output "T.png"
set datafile commentschars "#@"
pl 'nvt_temperature.xvg' u 1:2 w l
set output "potential.png"
pl 'nvt_potential.xvg' u 1:2 w l
unset output
exit
```

The desired behavior is fluctuation around an approximately stationary average rather than a persistent upward or downward drift.

## 10.3 Radial distribution functions: concept and use

The radial distribution function, $g(r)$, describes how the probability of finding one species changes as a function of distance from a reference species.

For two species A and B:

- $g_{AB}(r) \approx 0$: B is rarely found at distance $r$ from A;
- $g_{AB}(r) > 1$: B is enriched at that distance;
- a strong first peak usually corresponds to the first local coordination shell;
- the first minimum after the peak is often used to define the approximate end of that shell;
- $g(r) \to 1$ at large distance in a homogeneous bulk liquid.

Two types of RDF are calculated in this activity:

### Molecular RDFs

- $Li^+$ to FSI- center of mass
- $Li^+$ to DME center of mass

These describe where entire molecules are located relative to $Li^+$.

### Atomic RDFs

- $Li^+$ to oxygen atoms of $FSI^-$

- $Li^+$ to oxygen atoms of DME

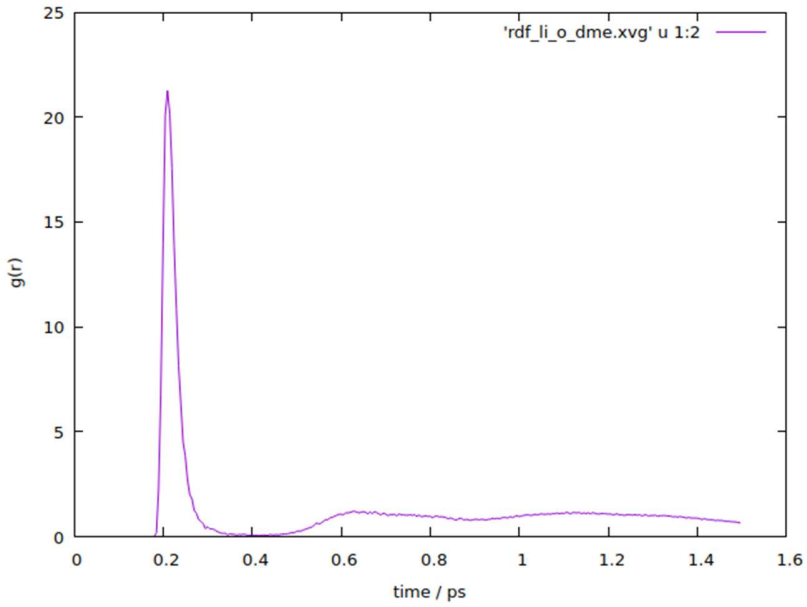
These are more directly connected to $Li^+$ coordination because oxygen atoms are the coordinating sites. RDFs are particularly useful in liquid electrolytes because they resolve local solvation and ion association. Li-O(DME) and Li-O(FSI) RDFs probe coordinating oxygen atoms around $Li^+$, whereas Li-DME and Li-FSI center-of-mass RDFs describe the spatial organization of whole solvent molecules and anions around $Li^+$.

```
gmx rdf -f nvt.xtc -s nvt.tpr \
    -ref 'name Li' \
    -sel 'resname DME and name "O*"' \
    -o rdf_li_o_dme.xvg \
    -rmax 1.2 -bin 0.005
```

Plot the rdf:

```
gnuplot
set terminal pngcairo enhanced size 1200,800
set datafile commentschars "#@"
set output "'rdf_li_o_dme.png"
pl 'rdf_li_o_dme.xvg' u 1:2 w l title 'Li-O(DME)'
unset output
exit
```

Example:




## 10.4 Practical equilibration checklist

Before continuing with the quantitative analysis, verify that:

- the NPT density has stabilized around a reasonable value;
- the NVT temperature fluctuates around the target temperature;
- the NVT potential energy does not show a systematic long-term drift;
- the trajectory looks physically reasonable in VMD;
- early and late RDFs are broadly consistent;

No single graph proves convergence. Taken together, these checks provide much stronger evidence that the part of the trajectory used for analysis represents an equilibrated liquid.

# SESSION 2 - Visualize and analyze the trajectory

## 1. Select the trajectory for analysis

Two trajectory sources are available for Session 2.

#### Option A - use the trajectory generated in Session 1

If Session 1 completed successfully, analyze:

```
session-1/nvt.tpr
session-1/nvt.xtc
session-1/nvt.edr
session-1/nvt_out.gro
```

#### Option B - use the supplied reference trajectory

If the Session 1 run did not finish, use the supplied reference files:

```
session-2/trajectory/nvt.tpr
session-2/trajectory/nvt.xtc
session-2/trajectory/nvt.edr
session-2/trajectory/nvt.gro
```

## 2. Load and visualize the NVT trajectory in VMD

### 2.1 Download the trajectory files

For a trajectory stored in Codespaces, use the file explorer to download:

```
nvt.gro
nvt.xtc
```

The .gro file supplies the initial structure and atom/residue names; the .xtc contains the trajectory frames.

### 2.2 Load the trajectory

1. Start VMD.
2. Choose **File > New Molecule**.
3. Browse to nvt.gro.
4. Click **Load**.
5. With the same molecule selected, choose **Load Data Into Molecule**.

6. Select nvt.xtc.
7. Click **Load**.

The animation controls at the bottom of the VMD main window provide play, pause, frame-by-frame navigation, and direct movement through the trajectory.

## 2.3 Display the periodic simulation box

Open:

**Extensions > Tk Console**

In the Tk Console type:

```
package require pbctools
pbc box
```

A box should appear around the system. This is the periodic boundary condition.

### Suggested representations

Create separate representations for Li+, FSI-, and DME.

### Observe

- Do $Li^+$ ions remain attached to one DME molecule for the complete trajectory?
- Do $FSI^-$ anions enter and leave the neighborhood of $Li^+$?
- How frequently do the local environments around $Li^+$ change?
- Can visual inspection alone quantify coordination? Why or why not?

# 3. Make a trajectory movie in VMD

1. Prepare the view and representations before rendering.
2. Display the periodic box if desired.
3. Open **Extensions > Visualization > Movie Maker**.
4. Select a trajectory-based movie rather than a simple camera rotation.
5. Choose the trajectory frame range.
6. For a quick classroom movie, use the OpenGL/Snapshot renderer if available; for higher quality, use an internal ray-tracing option such as Tachyon when supported by the local VMD installation.
7. Choose an available movie format and output name.
8. Start rendering.

Video encoding options vary slightly between VMD installations. If direct movie encoding is unavailable, VMD can render image frames which can later be combined with an external video encoder.

If VMD produces a sequence of snapshot images instead of a ready-made movie, the frames can be combined from a terminal with FFmpeg. First inspect the generated filenames and extension, then use the matching command.

```
ffmpeg -framerate 30 -i frame.%04d.tga -c:v libx264 -crf 18 -pix_fmt yuv420p trajector
y.mp4
```

For PNG snapshots, replace the input extension accordingly:

```
ffmpeg -framerate 30 -i frame.%04d.png -c:v libx264 -crf 18 -pix_fmt yuv420p trajector
y.mp4
```

The pattern %04d corresponds to sequential frame numbers such as frame.0000, frame.0001, frame.0002, etc. If VMD uses a different naming pattern, the FFmpeg input pattern must be adapted. FFmpeg must be installed on the computer where the snapshots are stored.

## 4. RDF analysis workflow

The RDF concept and its use as a structural stationarity check were introduced in Section 10.3 of Session 1. The following steps calculate full-trajectory molecular and atomic RDFs for quantitative analysis.

### Li-FSI center-of-mass RDF

Use one FSI center of mass per FSI residue:

```
gmx rdf -f nvt.xtc -s nvt.tpr \
    -ref 'name Li' \
    -sel 'res_com of resname FSI' \
    -o rdf_li_fsi_com.xvg \
    -rmax 1.2 -bin 0.005
```

### Li-DME center-of-mass RDF

```
gmx rdf -f nvt.xtc -s nvt.tpr \
    -ref 'name Li' \
    -sel 'res_com of resname DME' \
    -o rdf_li_dme_com.xvg \
    -rmax 1.2 -bin 0.005
```

### Li-O(FSI) RDF and coordination number

All FSI oxygen atom names beginning with O are selected:

```
gmx rdf -f nvt.xtc -s nvt.tpr \
    -ref 'name Li' \
    -sel 'resname FSI and name "O*"' \
    -o rdf_li_o_fsi.xvg \
    -cn cn_li_o_fsi.xvg \
    -rmax 1.2 -bin 0.005
```

`rdf_li_o_fsi.xvg` contains $g(r)$.
`cn_li_o_fsi.xvg` contains the cumulative number of selected oxygen atoms around Li+.

### Li-O(DME) RDF and coordination number

```
gmx rdf -f nvt.xtc -s nvt.tpr \
    -ref 'name Li' \
    -sel 'resname DME and name "O*"' \
    -o rdf_li_o_dme.xvg \
    -cn cn_li_o_dme.xvg \
    -rmax 1.2 -bin 0.005
```

This is an improved version of selecting only one atom such as name O1: selecting O* includes all DME oxygen atoms whose names begin with O.

## 4.1 Plot the RDFs with Gnuplot

```
gnuplot

set datafile commentschars "#@"
set terminal pngcairo size 1200,800
set output "rdf_molecular.png"
set xlabel "r (nm)"
set ylabel "g(r)"
set grid
plot "rdf_li_fsi_com.xvg" using 1:2 with lines title "Li-FSI COM", \
     "rdf_li_dme_com.xvg" using 1:2 with lines title "Li-DME COM"
unset output
exit
```

Open rdf_molecular.png from the Codespaces file explorer. Perform the same task to compare oxygen RDF.

To inspect coordination numbers, plot the corresponding cumulative curves:

```
gnuplot

set datafile commentschars "#@"
set terminal pngcairo size 1200,800
set output "coordination_numbers.png"
set xlabel "r (nm)"
set ylabel "Cumulative coordination number"
set grid
plot "cn_li_o_fsi.xvg" using 1:2 with lines title "O(FSI) around Li", \
     "cn_li_o_dme.xvg" using 1:2 with lines title "O(DME) around Li"
unset output
exit
```

Use the first minimum of each corresponding RDF as an approximate first-shell cutoff and read the cumulative coordination number at that distance.

### Discussion questions

1. Is the first Li-O(DME) or Li-O(FSI) peak closer to Li+?
2. Which oxygen type contributes more strongly to the first coordination shell?
3. How does the molecular Li-FSI RDF differ from the atomic Li-O(FSI) RDF?
4. Why is an oxygen RDF generally more useful than a whole-molecule COM RDF for discussing Li+ coordination?
5. Compare the qualitative conclusions with the solvation discussion in the reference article.

# 5. Mean-square displacement and diffusion

For normal three-dimensional diffusion, the Einstein relation is:

$$\langle |\mathbf{r}(t) - \mathbf{r}(0)|^2 \rangle = 6Dt$$

where:

- MSD is the mean-square displacement;
- $D$ is the self-diffusion coefficient;

- a linear MSD region is required for a meaningful diffusion fit.

`gmx msd` calculates the MSD and obtains $D$ from a linear regression.

By default, when -beginfit and -endfit are not supplied, GROMACS fits from 10% to 90% of the available MSD time range. This automatic choice is convenient but is not guaranteed to correspond to the best diffusive regime for every trajectory. The MSD curve should therefore be inspected before the automatically fitted diffusion coefficient is accepted.

## 5.1 $Li^+$ diffusion coefficient: first approximation

Run the MSD calculation using the default fit:

```
gmx msd -f nvt.xtc -s nvt.tpr -sel "name Li" -o msd_li.xvg
```

GROMACS reports a diffusion coefficient in units of: `1e-5 cm^2/s`

For example:

$$D = 0.25 \times 10^{-5}\ \mathrm{cm^2\,s^{-1}} = 0.25 \times 10^{-9}\ \mathrm{m^2\,s^{-1}}.$$

This first value should not be recorded as the final result before the MSD curve is inspected.

### Visualize the Li+ MSD and choose the fitting interval

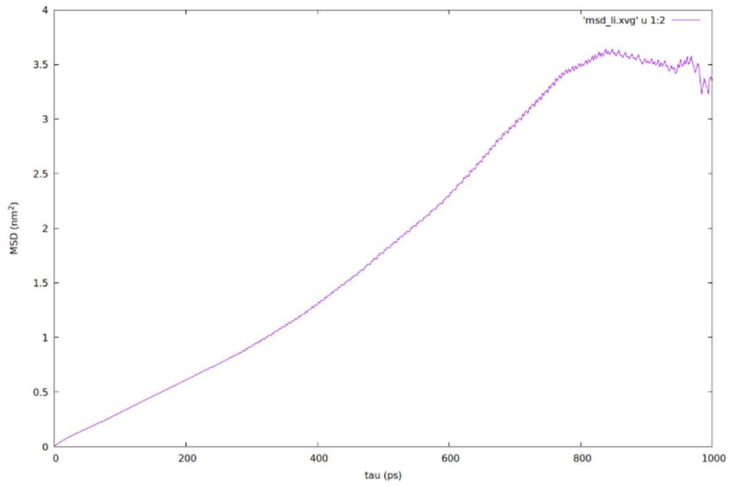
Start Gnuplot:

```
gnuplot
set datafile commentschars "#@"
set terminal pngcairo size 1200,800
set output "msd_Li.png"
pl 'msd_li.xvg' u 1:2 w l
unset output
exit
```

Inspect the shape of the curve.

At very short times, molecular motion may not yet be in the normal diffusive regime. At long lag times, the curve may become noisy because fewer statistically independent time origins contribute.

Example:




Choose a region that is approximately linear.

For the trajectory used in this exercise, 100-400 ps is an example fitting interval if that interval appears linear in the MSD plot:

```
gmx msd -f nvt.xtc -s nvt.tpr -sel "name Li" -o msd_li.xvg -beginfit 100 -endfit 400
```

This command recalculates the diffusion coefficient using only that fitting interval.

For FSI-, the translational motion of the molecular center of mass is required rather than the independent displacement of all FSI atoms. Therefore:

```
gmx msd -f nvt.xtc -s nvt.tpr \
    -sel 'res_com of resname FSI' \
    -o msd_fsi.xvg -beginfit 100 -endfit 400
```

### Discussion

1. Which species diffuses faster?
2. Is the difference large or small?
3. Is the MSD linear over the whole trajectory?
4. Why can the first part of the MSD deviate from linear diffusive behavior?
5. Why does the long-lag-time part of an MSD often become noisy?
6. How much does the calculated diffusion coefficient change when the automatic 10%-90% fit is replaced by the selected interval?
7. Why would a longer trajectory improve the diffusion estimate?

## 6. Estimate ionic conductivity: Nernst-Einstein relation

For a 1:1 monovalent electrolyte with Li+ and FSI-, the uncorrelated Nernst-Einstein conductivity is:

$$\sigma_{\mathrm{NE}} = \frac{F^2 c}{RT}(D_{\mathrm{Li}} + D_{\mathrm{FSI}}).$$

where:

- $F = 96485.33212\ \mathrm{C\,mol^{-1}}$,
- $R = 8.314462618\ \mathrm{J\,mol^{-1}\,K^{-1}}$,
- $T$ is the simulated temperature in K,
- $c$ is the LiFSI concentration in mol/m$^3$,
- $D_{Li}$ and $D_{FSI}$ are in m$^2$/s.

The result is in S/m units.

The Nernst-Einstein equation allows to estimate an approximate conductivity, given that it assumes that cations and anions diffuse independently. However, real electrolytes contain correlated motion such as $Li^+$ and $FSI^-$ moving together temporarily or oppositely charged ions forming contact ion pairs. Collective ion motion can therefore differ from the conductivity predicted using only self-diffusion coefficients.

## 7. Final results table

Complete this table before finishing the practical.

| Property | Result |
|---|---|
| DME molecules | 100 |
| $Li^+$ ions | 11 |
| $FSI^-$ ions | 11 |
| Equilibrated density / g mL$^{-1}$ | |
| Simulated temperature / K | |
| Simulated concentration / mol L$^{-1}$ | |
| Li-O(DME) first RDF peak / nm | |
| Li-O(FSI) first RDF peak / nm | |
| Li-O(DME) first-shell coordination number | |
| Li-O(FSI) first-shell coordination number | |
| Selected MSD fit interval / ps | |
| $D_{Li}$ / m$^2$ s$^{-1}$ | |
| $D_{FSI}$ / m$^2$ s$^{-1}$ | |
| $\sigma_{NE}$ / mS cm$^{-1}$ | |